



Distribution-Based Models

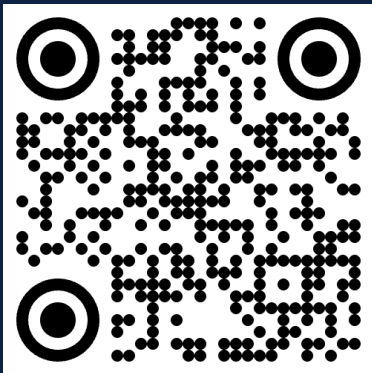
The Advanced R Series: Clustering & Classification

Slides and code created by Jesse Ghashti

February 2, 2026

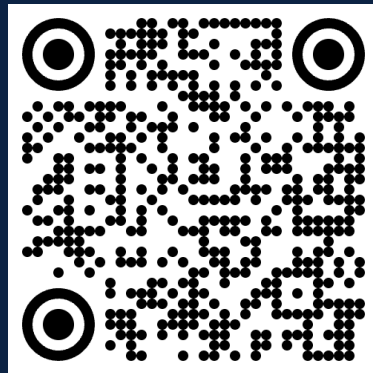
Centre for Scholarly Communication

The University of British Columbia | Okanagan Campus | Syilx Okanagan Nation Territory



←
Check out our other
CSC workshops!

GitHub code and
slides for today's
workshop (and pre-
vious workshops)



Alternatively, code/slides available at the bottom of
<https://csc-ubc-okanagan.github.io/workshops/>

Workshop Series Overview



Session	Topic	Date/Time
1	Hierarchical Models	Jan 14, 12:00 PM
2	Centroid-Based Models	Jan 19, 3:30 PM
3	Fuzzy Clustering	Jan 28, 1:00 PM
4	Distribution-Based Models	Feb 2, 3:30 PM
5	Density-Based Models	Feb 9, 3:30 PM
6	Graph-Based Models	Feb 23, 3:30 PM
7	Mixed-Type Data Quantification	Mar 4, 1:00 PM
8	Mixed-Type Data Clustering	Mar 11, 1:00 PM
9	Bias Reduction and Fairness	Mar 18, 1:00 PM
10	Dimensionality Reduction 1 of 2	Mar 23, 3:30 PM
11	Dimensionality Reduction 2 of 2	Mar 30, 3:30 PM

What we learned...

- Learned fuzzy C -means (FCM) and soft membership assignments
- Explored the effect of the fuzzifier parameter m
- Introduced fuzzy validity metrics: PC, PE, and Xie-Beni index
- Compared hard (K -means) vs soft (FCM) clustering
- Applied FCM to image segmentation with uncertainty maps
- Discussed how membership uncertainty reveals ambiguous boundaries

Today -> Distribution-Based Clustering



Today we will...

- Understand the limitations of K -means for non-spherical clusters
- Learn Gaussian Mixture Models (GMMs) and the EM algorithm
- Explore covariance structures and confidence ellipses
- Use BIC and ICL for model selection
- Introduce Dirichlet Process Mixture Models (DPMM)
- Apply GMM to the Wisconsin Breast Cancer dataset

Today we require...

```
library('mclust')           # Gaussian mixture models
library('mixtools')         # mixture model tools
library('ggplot2')          # visualization
library('dplyr')            # data manipulation
library('gridExtra')        # plot arrangement
library('MASS')             # multivariate normal simulation
library('mvtnorm')          # multivariate normal densities
library('dirichletprocess') # Dirichlet process
```

A Reminder of the Limitations of K -means



K -means assumes spherical clusters

K -means minimizes WCSS using Euclidean distance, which implicitly assumes...

- Clusters are roughly spherical
- Clusters have similar sizes and variances
- No cluster-specific shape or orientation

But real data often has

- Elongated or elliptical clusters
- Different variances per cluster
- Correlated features within clusters

So we ask... what if each cluster has its own mean (centroid) **AND covariance**?

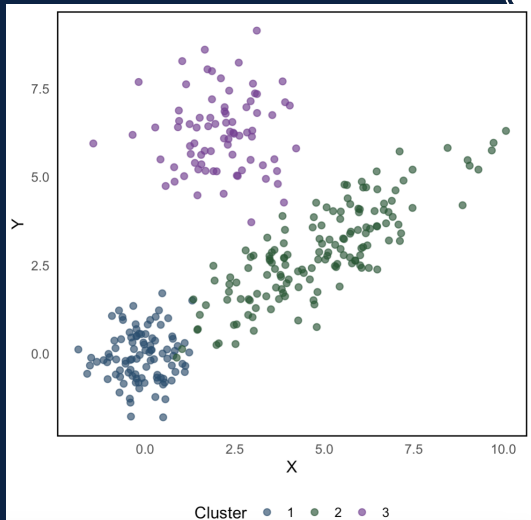
Data with Different Covariances



```
set.seed(2026)
cluster1 <- mvrnorm(100, mu = c(0, 0),
                    Sigma = diag(2) * 0.5)
sigma2 <- matrix(c(4, 2.5, 2.5, 2), nrow = 2)
cluster2 <- mvrnorm(150, mu = c(5, 3),
                    Sigma = sigma2)
cluster3 <- mvrnorm(80, mu = c(2, 6),
                    Sigma = diag(2) * 1.5)

syntheticData <- data.frame(
  x = c(cluster1[,1], cluster2[,1], cluster3[,1]),
  y = c(cluster1[,2], cluster2[,2], cluster3[,2]),
  trueCluster = factor(c(rep(1, 100),
                          rep(2, 150),
                          rep(3, 80))))

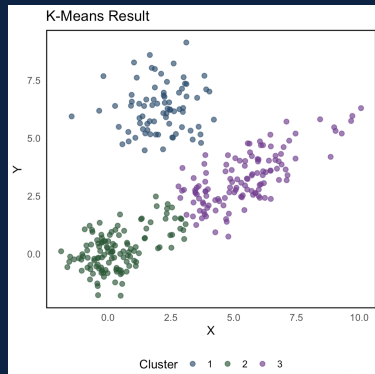
ggplot(syntheticData, aes(x = x, y = y,
                          color = trueCluster)) +
  geom_point(alpha = 0.7, size = 2) +
  scale_color_manual(values = c("#1B4F72",
                                "#145A32", "#7D3C98"))
```



K-means on Non-Spherical Clusters



```
kmeansResult <- kmeans(  
  syntheticData[, c("x", "y")],  
  centers = 3, nstart = 25)  
syntheticData$kmeansCluster <-  
  factor(kmeansResult$cluster)  
  
ggplot(syntheticData,  
  aes(x = x, y = y,  
    color = kmeansCluster)) +  
  geom_point(alpha = 0.7, size = 2) +  
  scale_color_manual(values = c("#1B4F72",  
    "#145A32", "#7D3C98"))
```



K-means cannot capture the elongated shape of one of the clusters because it uses Euclidean distance. Therefore, points near the boundary are assigned 'incorrectly'.

Modelling Data as a Mixture of Gaussians

Data is generated from a mixture of K multivariate normal distributions, each with its own mean μ_k and covariance Σ_k .

Density Function

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$

- π_k = mixing proportion (weight) of component k , where $\sum_k \pi_k = 1$
- μ_k = mean vector of component k
- Σ_k = covariance matrix of component k
- $\mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$ = multivariate normal density

Each cluster can have its own shape, size, and orientation through Σ_k

From Densities to Cluster Memberships

Posterior probability that observation $\mathbf{x}_i$ belongs to cluster k is given by

$$\gamma_{ik} = P(z_i = k \mid \mathbf{x}_i) = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

- $\gamma_{ik} \in [0, 1]$ and $\sum_{k=1}^K \gamma_{ik} = 1$ for each i
- Similar to fuzzy memberships from Workshop 3!
- Hard assignment: $\hat{k}_i = \arg \max_k \gamma_{ik}$
- Uncertainty: $1 - \max_k \gamma_{ik}$

The EM Algorithm (Yet Another Iterative Algorithm for our Toolbox)



Estimation takes place through Expectation-Maximization (EM) Algorithm

E-step (Expectation) Compute posterior probabilities using current parameters

$$\gamma_{ik}^{(t)} = \frac{\pi_k^{(t)} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k^{(t)}, \boldsymbol{\Sigma}_k^{(t)})}{\sum_{j=1}^K \pi_j^{(t)} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j^{(t)}, \boldsymbol{\Sigma}_j^{(t)})}$$

M-step (Maximization) Update parameters using posteriors

$$\boldsymbol{\mu}_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik}^{(t)} \mathbf{x}_i}{\sum_{i=1}^n \gamma_{ik}^{(t)}}, \quad \pi_k^{(t+1)} = \frac{1}{n} \sum_{i=1}^n \gamma_{ik}^{(t)}$$

$$\boldsymbol{\Sigma}_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik}^{(t)} (\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})(\mathbf{x}_i - \boldsymbol{\mu}_k^{(t+1)})^\top}{\sum_{i=1}^n \gamma_{ik}^{(t)}}$$

Repeat until log-likelihood converges.

EM Algorithm, K -means, Fuzzy C -means Comparison



	K -means	Fuzzy C -means	GMM (EM)
Assignment	Hard (0/1)	Soft (matrix $\mathbf{U}$)	Soft (posterior probs)
Cluster shape	Spherical	Spherical	Elliptical (any shape)
Parameters	Centroids only	Centroids & m	μ_k, Σ_k, π_k
Objective	Minimize WCSS	Minimize WCSS	Maximize log-likelihood
Update rule	Assign & re-centre	Assign & re-centre	E-step & M-step
Uncertainty	None	Membership Matrix $\mathbf{U}$	Posterior probabilities

Interestingly, K -means is a special case of GMM (the details of which I leave to the interested individuals to research).

```
library(mclust)

gmmAuto <- Mclust(syntheticData[, c("x", "y")])
gmmAuto$modelName
gmmAuto$G
round(gmmAuto$loglik, 2)
round(gmmAuto$bic, 2)

gmm3 <- Mclust(syntheticData[, c("x", "y")], G = 3)
syntheticData$gmmCluster <-
  factor(gmm3$classification)

posteriorProbs <- gmm3$z
colnames(posteriorProbs) <-
  paste0("pclust", 1:ncol(posteriorProbs))

syntheticData$gmmMaxProb <-
  apply(posteriorProbs, 1, max)

round(gmm3$parameters$pro, 4)
round(gmm3$parameters$mean, 4)

for (k in 1:3) {
  cat("\nCluster", k, ":\n")
  print(round(
    gmm3$parameters$variance$sigma[,k], 4))
}
```

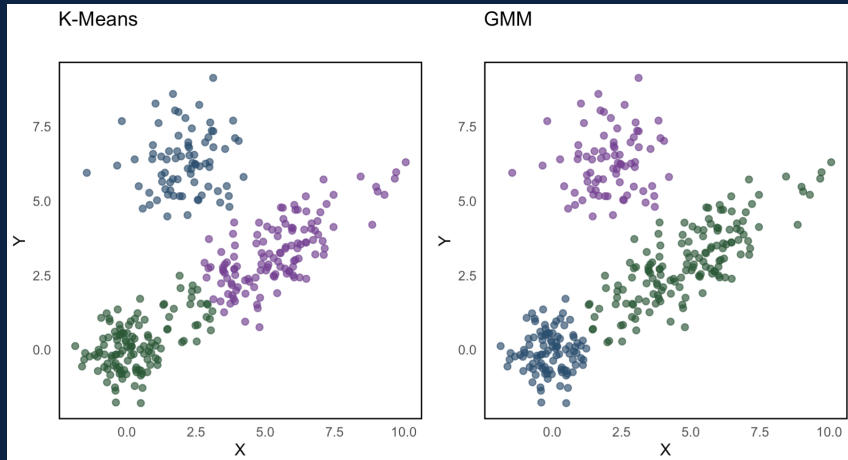
`mclust` automatically

- Selects number of components via BIC
- Tests different covariance structures
- Returns posterior probabilities (`$z`)
- Estimates all GMM parameters

Some important outputs include

- `$parameters$pro`: mixing proportions π_k
- `$parameters$mean`: centroids μ_k
- `$parameters$variance$sigma`: covariances Σ_k
- `$z`: posterior probability matrix

K -means vs GMM

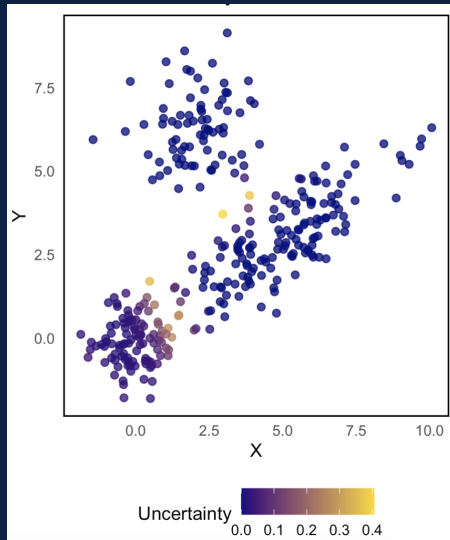


GMM Uncertainty



```
syntheticData$uncertainty <-  
  1 - syntheticData$gmmMaxProb  
  
ggplot(syntheticData,  
  aes(x = x, y = y,  
    color = uncertainty)) +  
  geom_point(alpha = 0.8, size = 2) +  
  scale_color_gradient(low = "navy",  
    high = "gold",  
    name = "Uncertainty") +  
  labs(title = "GMM Uncertainty",  
    x = "X", y = "Y")
```

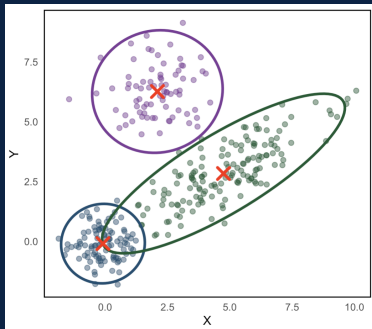
Just like FCM, uncertainty is highest at cluster boundaries where posterior probabilities are split



Each component's covariance Σ_k defines an ellipse through eigendecomposition:

- Eigenvectors determine orientation, i.e., axes of ellipse
- Eigenvalues determine spread along each axis
- Scaled by $\chi^2_2(0.95)$ for a 95% confidence region

```
generateEllipse <- function(mu, sigma,
                             nPoints = 100,
                             level = 0.95) {
  eigenDecomp <- eigen(sigma)
  eigenValues <- eigenDecomp$values
  eigenVectors <- eigenDecomp$vectors
  chiSqVal <- qchisq(level, df = 2)
  theta <- seq(0, 2 * pi, length.out = nPoints)
  circle <- cbind(cos(theta), sin(theta))
  ellipse <- circle %*%
    diag(sqrt(eigenValues * chiSqVal)) %*%
    t(eigenVectors)
  ellipse <- sweep(ellipse, 2, mu, "+")
  return(as.data.frame(ellipse))
}
```



Parameterization of Σ_k

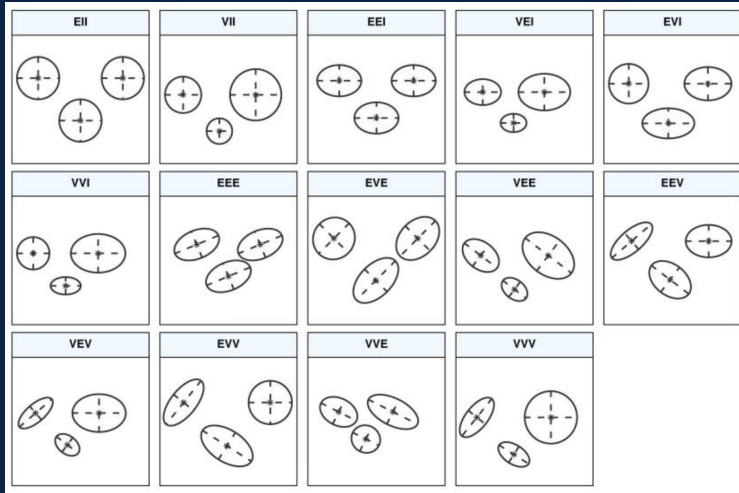
`mclust` decomposes covariance as $\Sigma_k = \lambda_k \mathbf{D}_k \mathbf{A}_k \mathbf{D}_k^\top$ where:

- λ_k = volume (size of cluster)
- $\mathbf{D}_k$ = orientation (eigenvectors)
- $\mathbf{A}_k$ = shape (ratio of eigenvalues)

Common models include

- **EII**: Equal volume, spherical (similar to K -means)
- **VII**: Variable volume, spherical
- **EEE**: Equal volume, shape, orientation
- **VVV**: Variable volume, shape, orientation (most flexible)

So Many Models!!



Source: Scrucca L, Fop M, Murphy TB, Raftery AE. mclust 5: Clustering, Classification and Density Estimation Using Gaussian Finite Mixture Models. R J. 2016. 8(1): 289-317.

Choosing the Number of Components

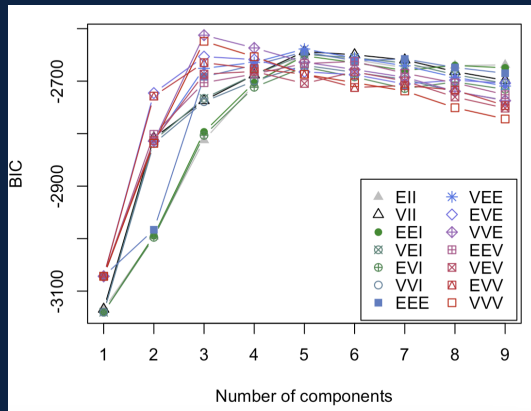


BIC for GMM Model Selection

$$\text{BIC} = -2 \cdot \ell(\hat{\theta}) + k \cdot \log(n)$$

- $\ell(\hat{\theta})$ = maximized log-likelihood
- k = number of free parameters
- n = sample size
- Higher BIC is better in `mclust` (different sign convention)

```
bicValues <- mclustBIC(syntheticData[, c("x", "y")], G = 1:9)
summary(bicValues)
plot(bicValues)
```

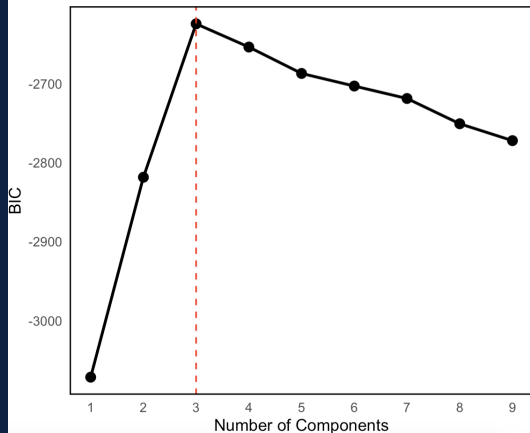


```
bicVVV <- bicValues[, "VVV"]
bicDf <- data.frame(k = 1:9, BIC = bicVVV)

ggplot(bicDf, aes(x = k, y = BIC)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_vline(xintercept = which.max(bicVVV),
            linetype = "dashed",
            color = "red") +
  labs(title = "BIC for VVV Model",
       x = "Number of Components",
       y = "BIC") +
  scale_x_continuous(breaks = 1:9)
```

ICL (Integrated Complete-data Likelihood) additionally penalizes for entropy of soft assignments

BIC for VVV Model (Flexible Covariance)



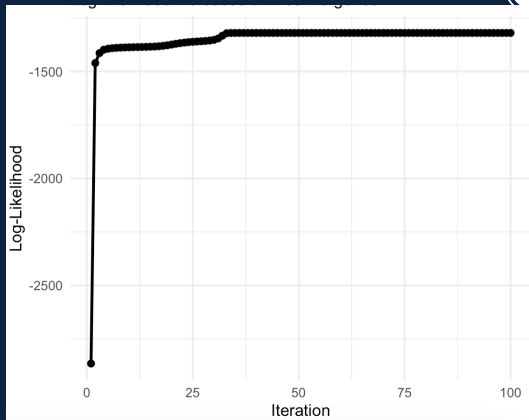
Visualizing EM Convergence



```
library(mixtools)
dataMatrix <- as.matrix(
  syntheticData[, c("x", "y")]
)
emResult <- mvnnormalmixEM(dataMatrix, k = 3,
  verb = TRUE,
  epsilon = 1e-6)

l1Df <- data.frame(
  iteration = 1:length(emResult$all.loglik),
  logLik = emResult$all.loglik)

ggplot(l1Df, aes(x = iteration, y = logLik)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(title = "EM Algorithm Convergence",
    x = "Iteration",
    y = "Log-Likelihood")
```



Log-likelihood is guaranteed to increase (or stay the same) at every EM iteration

GMM Metrics

Avg log-likelihood ($\overline{\text{LL}}$)

$$\frac{1}{n} \ell(\hat{\theta})$$

Entropy (ENT)

$$-\frac{1}{n} \sum_i \sum_k \gamma_{ik} \log \gamma_{ik}$$

Avg max posterior ($\overline{\text{MP}}$)

$$\frac{1}{n} \sum_i \max_k \gamma_{ik}$$

```
ariKmeans <- computeARI(syntheticData$trueCluster, syntheticData$kmeansCluster)
nmiKmeans <- computeNMI(syntheticData$trueCluster, syntheticData$kmeansCluster)
ariGmm <- computeARI(syntheticData$trueCluster, syntheticData$gmmCluster)
nmiGmm <- computeNMI(syntheticData$trueCluster, syntheticData$gmmCluster)

avgLogLik <- gmm3$loglik/nrow(syntheticData)
round(avgLogLik, 4)
entropy <- -mean(rowSums(posteriorProbs*log(posteriorProbs + 1e-10)))
round(entropy, 4)
avgMaxPost <- mean(syntheticData$gmmMaxProb)
round(avgMaxPost, 4)
```

	ARI	NMI	$\overline{\text{LL}}$	ENT	$\overline{\text{MP}}$
<i>K</i> -means	0.646	0.557	–	–	–
GMM	0.952	0.926	-3.836	0.076	0.977

When K is Truly Unknown

GMM requires selecting K via BIC, but what if the number of clusters is true unknown?

DPMM is a Bayesian nonparametric approach that infers K from the data.

- Potentially infinite number of mixture components
- Number of active clusters grows with data
- Controlled by concentration parameter α
- No need to pre-specify K

Although it sounds great, it is much slower than parametric GMM, as DPMM it uses MCMC sampling.

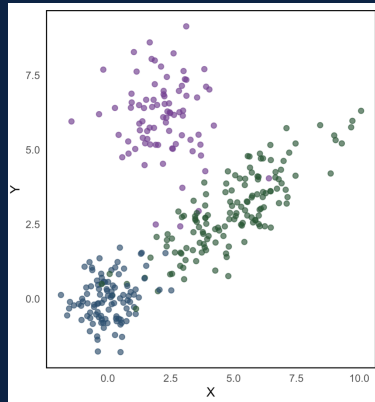
Fitting a DPMM



```
library(dirichletprocess)
scaledData <- scale(
  syntheticData[, c("x", "y")]
)
dpObj <- DirichletProcessMvnormal(
  scaledData, alphaPriors = c(4, 8))
dpFit <- Fit(dpObj, 500, progressBar = TRUE)

dpClusters <- dpFit$clusterLabels
syntheticData$dpmmCluster <- factor(dpClusters)
nClustersFound <- length(unique(dpClusters))
nClustersFound
table(dpClusters)

ariDpmm <- computeARI(
  syntheticData$trueCluster,
  syntheticData$dpmmCluster)
nmiDpmm <- computeNMI(
  syntheticData$trueCluster,
  syntheticData$dpmmCluster)
```



We should run more than 500 iterations for a reliable result, but DPMM is slow due to MCMC sampling. Even so, DPMM found 3 clusters with concentration parameter $\alpha = 4$.

Stick-Breaking Representation

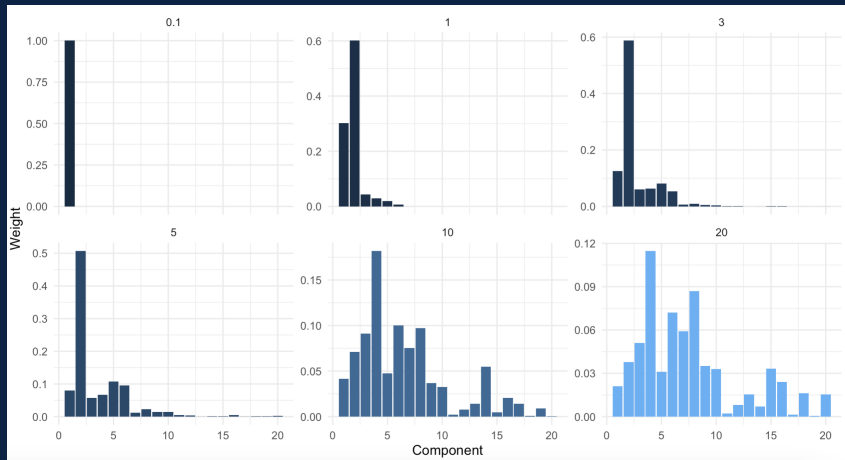
The Dirichlet Process generates cluster weights through **stick-breaking**

$$\beta_k \sim \text{Beta}(1, \alpha), \quad \pi_k = \beta_k \prod_{j=1}^{k-1} (1 - \beta_j)$$

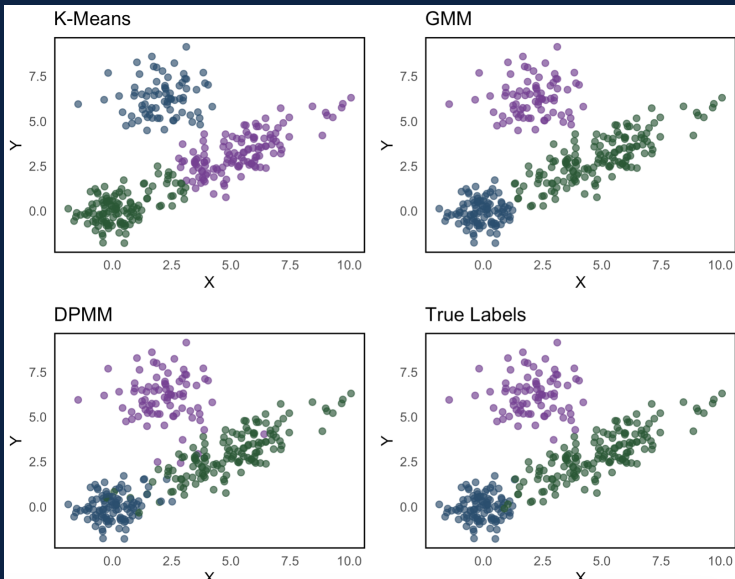
- α small: most mass on 1–2 clusters
- α moderate: spread across several clusters
- α large: many small clusters

```
simulateStickBreaking <- function(alpha, nDraws = 20) {  
  betas <- rbeta(nDraws, 1, alpha); weights <- numeric(nDraws); remaining <- 1  
  for (i in 1:nDraws) {  
    weights[i] <- betas[i] * remaining  
    remaining <- remaining * (1 - betas[i])  
  }  
  return(weights)  
}  
alphaValues <- c(0.1, 1.0, 5.0, 20.0)
```

Visualizing Stick-Breaking



Comparison of All Methods



Wisconsin Diagnostic Breast Cancer (WDBC)

Cluster tumour cell nuclei measurements into groups and compare with known diagnoses (Malignant or Benign).

Ten features computed for each cell nucleus include

1. Radius
2. Texture
3. Perimeter
4. Area
5. Smoothness
6. Compactness
7. Concavity
8. Concave points
9. Symmetry
10. Fractal dimension

For each feature: mean, standard deviation, and extreme value are recorded → 30 total features.

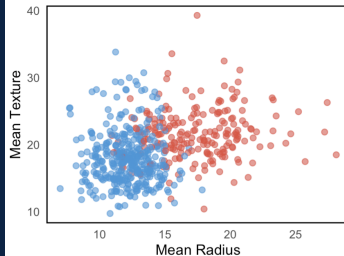
We have not discussed variable selection here. We are exploring 3 different potential options. Consider trying different combinations or researching variable selection techniques (future workshop).

```
data(wdbc)
Diagnosis <- wdbc[, 2]
features <- wdbc[, -c(1,2)]
table(Diagnosis)

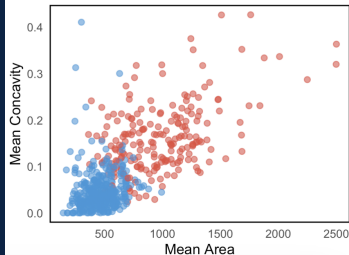
wdbcDf <- data.frame(
  Diagnosis = Diagnosis, features)

ggplot(wdbcDf, aes(x = Radius_mean,
                  y = Texture_mean,
                  color = Diagnosis)) +
  geom_point(alpha = 0.6, size = 2) +
  scale_color_manual(
    values = c("B" = "#3498DB",
              "M" = "#E74C3C"),
    labels = c("B" = "Benign",
              "M" = "Malignant"))
```

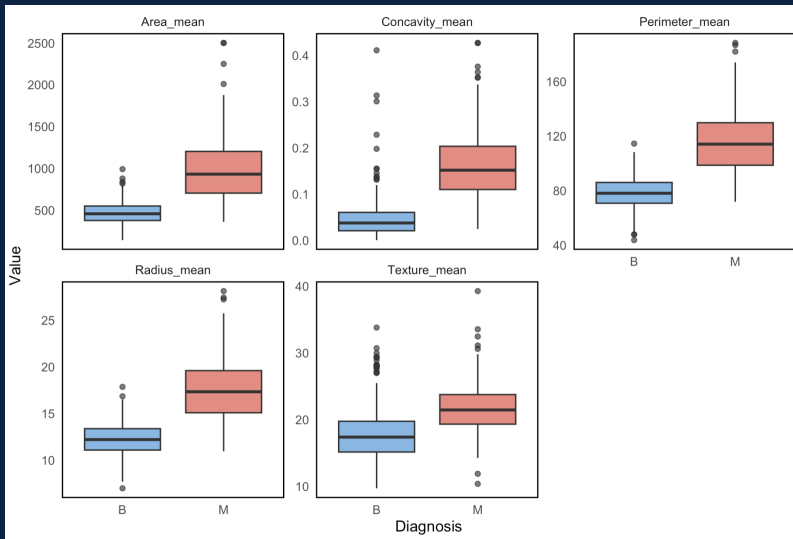
Radius vs Texture (Mean)



Area vs Concavity (Mean)



Feature Distributions by Diagnosis

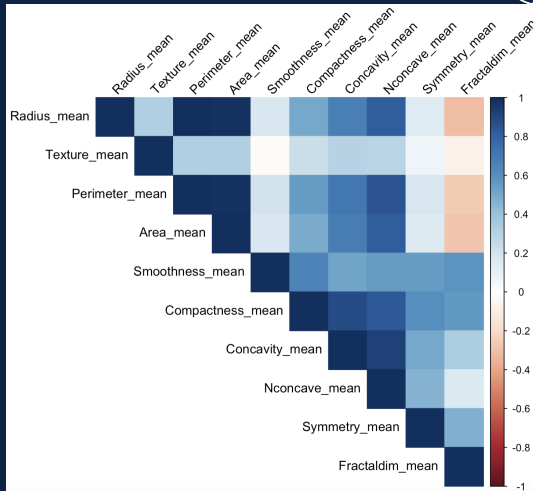


Feature Correlations



```
meanFeatures <- features[, 1:10]
corrMatrix <- cor(meanFeatures)
corrplot(corrMatrix, method = "color",
  type = "upper",
  tl.col = "black", tl.srt = 45,
  title = "Mean Features",
  mar = c(0, 0, 2, 0))
```

Many features are highly correlated.
For demonstration we use all features,
but reducing dimensionality may help.



```
featuresScaled <- scale(features)
meanFeaturesScaled <- scale(meanFeatures)

gmmBicAll <- mclustBIC(featuresScaled, G = 1:9)
gmmOptimalAll <- Mclust(featuresScaled,
                        x = gmmBicAll)
gmmOptimalAll$modelName
gmmOptimalAll$G

gmmBicMean <- mclustBIC(meanFeaturesScaled,
                        G = 1:9)
gmmOptimalMean <- Mclust(meanFeaturesScaled,
                         x = gmmBicMean)
gmmOptimalMean$modelName
gmmOptimalMean$G

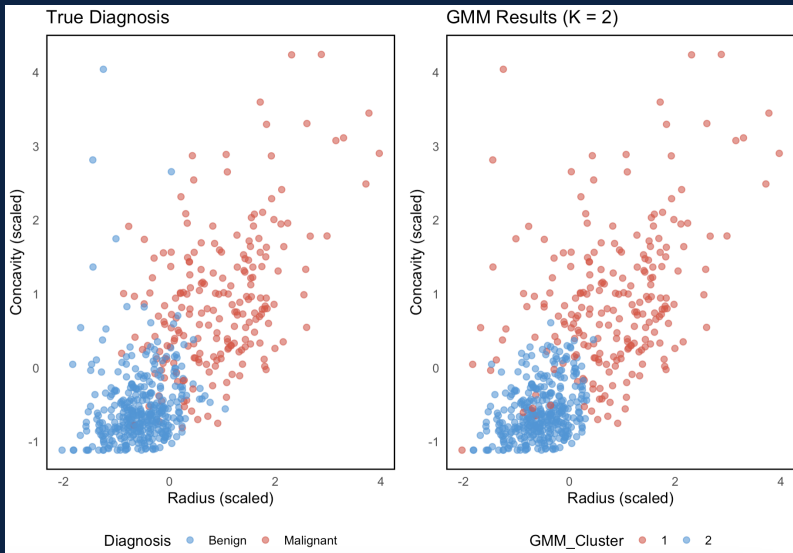
gmmK2 <- Mclust(meanFeaturesScaled, G = 2)
gmmK2$modelName
round(gmmK2$loglik, 2)
round(gmmK2$bic, 2)
```

Three approaches

1. All 30 features, auto K : BIC selects 7 clusters
2. 10 mean features, auto K : BIC selects 5 clusters
3. 10 mean features, $K = 2$: Match known diagnosis groups

BIC favours more clusters due to substructure within diagnosis groups. We move forward with $K = 2$ as it aligns with the classification task.

GMM Results and True Diagnosis



Posterior Probabilities and Uncertainty

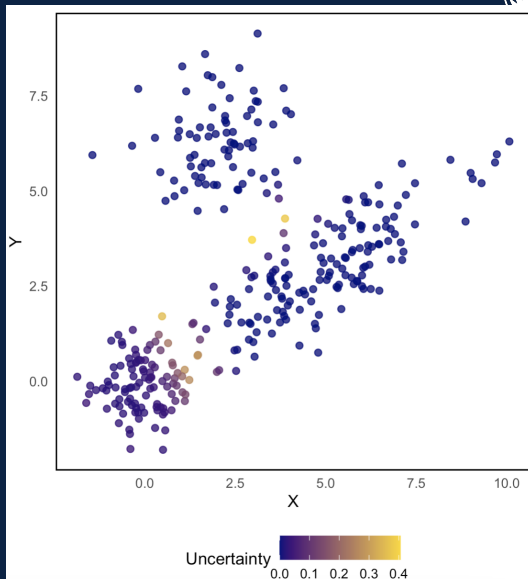


```
posteriorProbs <- gmmK2$z
colnames(posteriorProbs) <- c("pclust1", "pclust2")

resultsDf$pclust1 <- posteriorProbs[, 1]
resultsDf$pclust2 <- posteriorProbs[, 2]
resultsDf$maxPosterior <-
  apply(posteriorProbs, 1, max)
resultsDf$uncertainty <-
  1 - resultsDf$maxPosterior

round(mean(resultsDf$uncertainty), 4)
round(max(resultsDf$uncertainty), 4)

uncertainThreshold <- 0.8
uncertainCases <- resultsDf[
  resultsDf$maxPosterior < uncertainThreshold, ]
nrow(uncertainCases)
table(uncertainCases$Diagnosis)
```

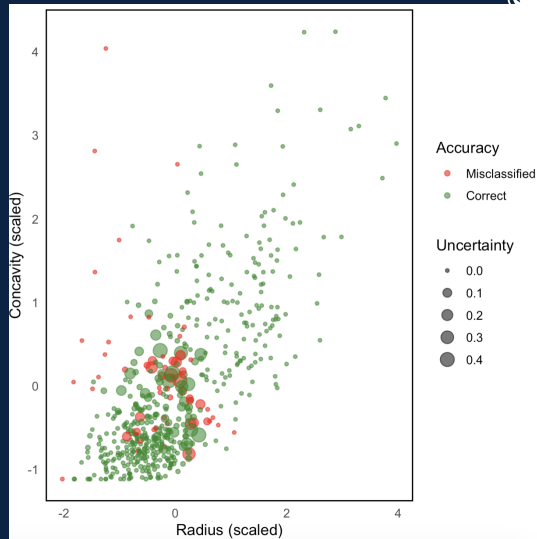


Misclassifications

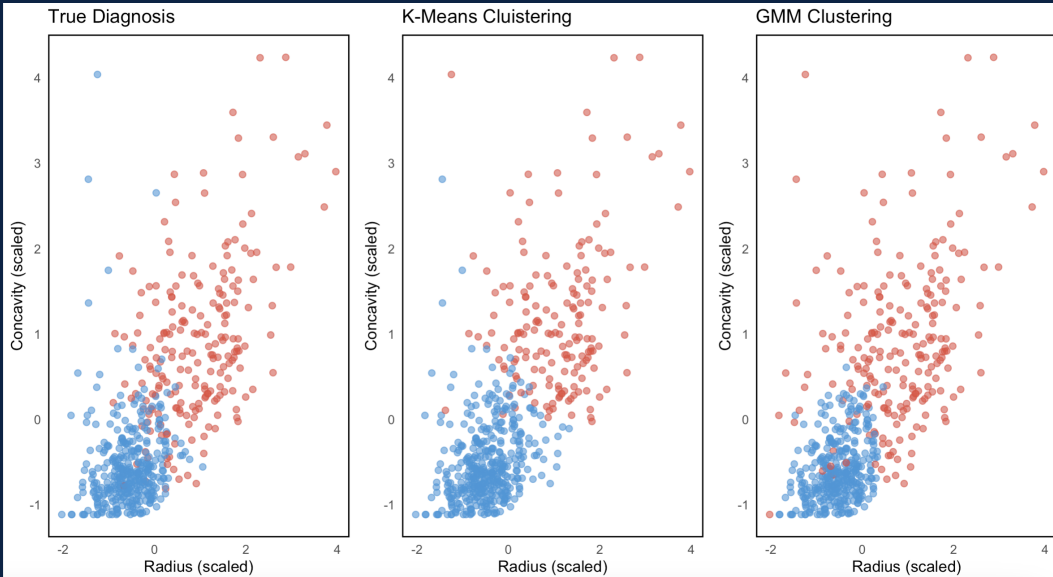


```
clusterToDiagnosis <- if (  
  sum(Diagnosis == "M" & gmmClustersK2 == 2) >  
  sum(Diagnosis == "M" & gmmClustersK2 == 1))  
  c("B", "M") else c("M", "B")  
  
resultsDf$predictedDiagnosis <-  
  clusterToDiagnosis[gmmClustersK2]  
resultsDf$correct <-  
  resultsDf$Diagnosis ==  
  resultsDf$predictedDiagnosis  
  
sum(resultsDf$correct)  
sum(!resultsDf$correct)  
  
misclassified <- resultsDf[!resultsDf$correct, ]  
sum(misclassified$Diagnosis == "B")  
sum(misclassified$Diagnosis == "M")
```

Misclassified points tend to have higher uncertainty, appearing near the decision boundary between clusters.



K -means vs GMM on WDBC



Method	ARI	NMI
K-means	0.6465	0.5570
GMM (EM)	0.6265	0.5064

K-means vs True Labels

True / K	1	2
1	351	6
2	49	163

GMM vs True Labels

True / GMM	1	2
1	322	35
2	24	188

K-means achieves higher agreement with the true labels (ARI, NMI). This is not the case when we do feature selection...

Feature	K-means (B)	K-means (M)	GMM (B)	GMM (M)
Radius_mean	-0.47	1.12	-0.55	0.83
Texture_mean	-0.22	0.51	-0.26	0.39
Perimeter_mean	-0.49	1.15	-0.56	0.86
Area_mean	-0.47	1.11	-0.54	0.82
Smoothness_mean	-0.28	0.66	-0.32	0.48
Compactness_mean	-0.46	1.08	-0.55	0.83
Concavity_mean	-0.51	1.20	-0.60	0.91
Nconcave_mean	-0.54	1.28	-0.62	0.95
Symmetry_mean	-0.26	0.62	-0.31	0.47
Fractaldim_mean	-0.08	0.18	-0.14	0.21

- Both methods learn similar splits as all features increase together from B to M .
- GMM means are less extreme than K-means on several features, which is often seen with soft assignment pulling means toward the overlap region.

Risk Groups from Posteriors



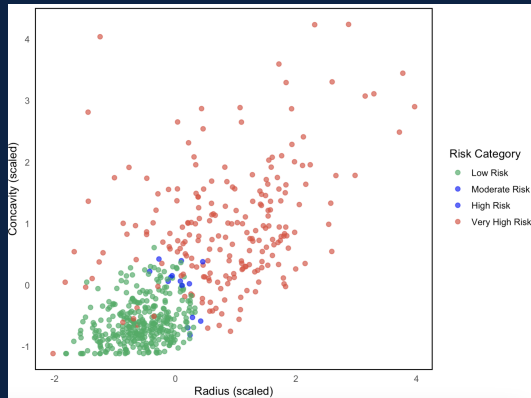
```
malignantCluster <- which.max(c(
  sum(Diagnosis == "M" & gmmClustersK2 == 1),
  sum(Diagnosis == "M" & gmmClustersK2 == 2)))

resultsDf$malignantProbability <-
  posteriorProbs[, malignantCluster]

resultsDf$riskCategory <- cut(
  resultsDf$malignantProbability,
  breaks = c(0, 0.2, 0.5, 0.8, 1),
  labels = c("Low Risk", "Moderate Risk",
             "High Risk", "Very High Risk"),
  include.lowest = TRUE)

riskTable <- table(resultsDf$riskCategory,
  resultsDf$Diagnosis)

riskTable
```



- GMMs model each cluster as a multivariate normal with its own μ_k and Σ_k
- EM algorithm iterates E-step (posteriors) and M-step (parameter updates)
- Covariance structures allow clusters of different shapes, sizes, and orientations
- BIC selects both the number of components and covariance model
- Posterior probabilities provide soft assignments and uncertainty, like FCM
- DPMM infers K from data via Bayesian nonparametrics (but is slower)
- Concentration parameter α controls prior expectation of cluster count

Next Workshop: Feb 9, 3:30 PM

Additional Questions?
Book an Appointment!



Density-Based Clustering

-> Workshop 2 (K -means) had circular cluster and hard partition limitations.

-> Workshop 3 (FCM) addressed the hard partition limitation.

-> Workshop 4 (GMM) addressed the circular cluster limitation.

-> Next workshop will address clusters of **arbitrary shape** using density connectivity.

Thank You!

Questions?

Workshop Materials:

<https://csc-ubc-okanagan.github.io/workshops/>

Contact:

jesse.ghashti@ubc.ca